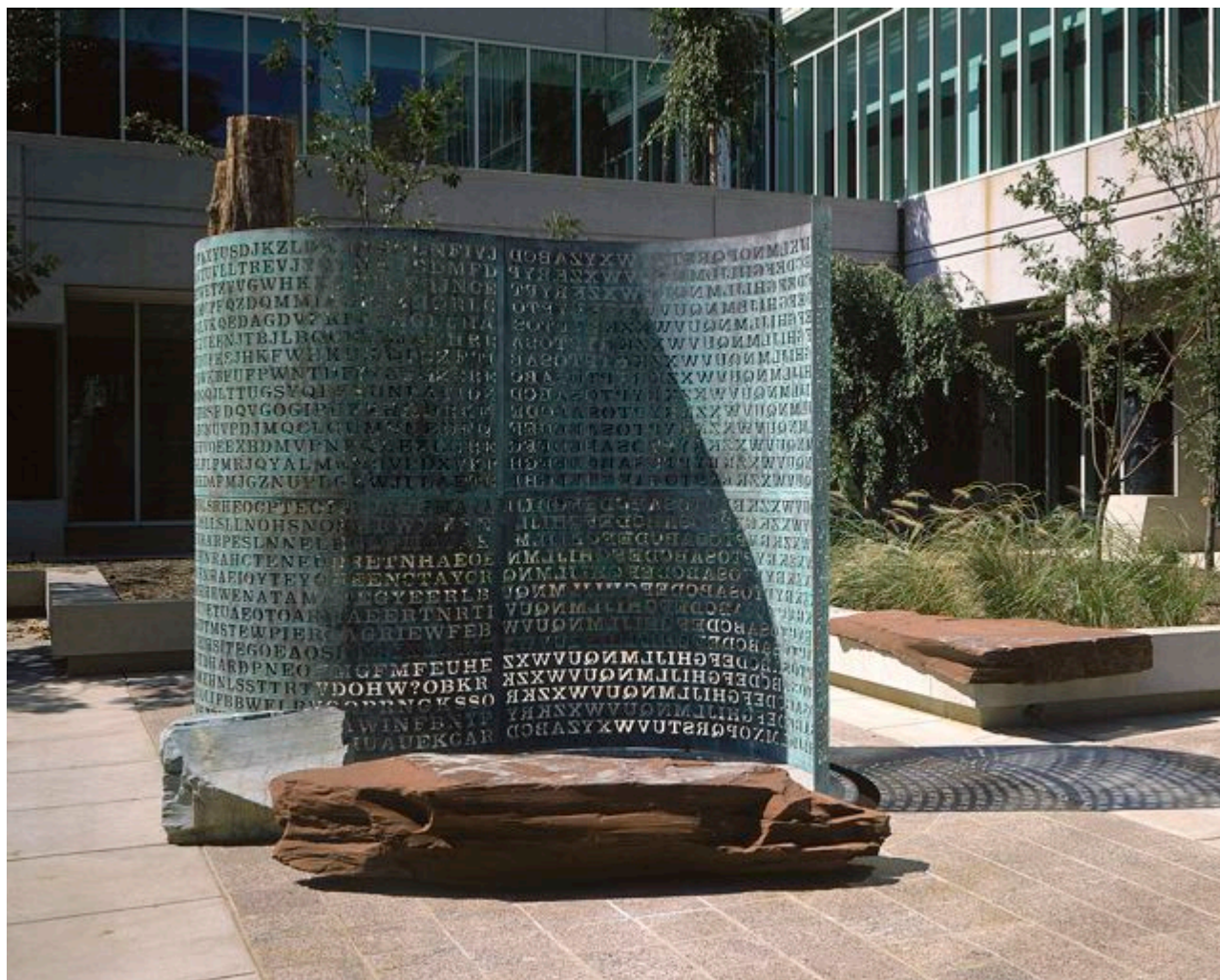


# FIN 4600 Lab: Pretty Good Privacy (PGP)

Michigan Technological University · College of Business · FIN 4600 Financial Technology Foundations



*Kryptos by Jim Sanborn, CIA Headquarters. Photo: Jim Sanborn, CC BY-SA 3.0.*

In this two-session lab you create your own OpenPGP key, check and certify your instructor's key, and exchange a signed, encrypted message with your instructor. These are the same steps banks and payment processors follow when they set up encrypted file transfers with a new counterparty.

**The one rule to remember:** encrypt with the recipient's **public** key. Sign with your own **private** key.

## What's in this repository

| Path                                                     | Contents                                                                   |
|----------------------------------------------------------|----------------------------------------------------------------------------|
| <a href="#">notebooks/FIN4600_PGP_Lab_Student.ipynb</a>  | The lab notebook, Parts A to E                                             |
| <a href="#">docs/study-guide.md</a>                      | Concepts, diagrams, key terms, self-check questions, and the lab checklist |
| <a href="#">docs/slides.md</a>                           | The lecture slides, with notes                                             |
| <a href="#">pyproject.toml</a> / <a href="#">uv.lock</a> | Python packages for the notebook, managed by uv                            |

| Path                       | Contents                                                 |
|----------------------------|----------------------------------------------------------|
| <a href="#">.gitignore</a> | Keeps keys, encrypted files, and your answers out of git |

## Before session 1

1. **Install GnuPG with a key manager.**
  - Windows: [Gpg4win](#) (includes Kleopatra)
  - macOS: [GPG Suite](#) (includes GPG Keychain)
2. **Install `uv`**, then create the virtual environment and install the packages from this folder:

```
uv sync
```

The package is `python-gnupg`. Do **not** install the unrelated package named `gnupg`.

3. **Read** [docs/study-guide.md](#).

Do not create your key pair yet. You will do that in class.

## Running the notebook

Run the notebook **on your own laptop**, not in Google Colab. It must use the same GnuPG keyring as Kleopatra or GPG Keychain.

```
uv run jupyter lab notebooks/FIN4600_PGP_Lab_Student.ipynb
```

VS Code with the Jupyter extension also works: open the notebook and pick the `.venv` created by `uv sync` in the kernel picker.

Save the files from Canvas ([instructor\\_pub.asc](#) in session 1, your [challenge\\_<user>.asc](#) in session 2) into the `notebooks/` folder, next to the notebook.

When GnuPG needs your passphrase, a separate window opens. If a cell seems stuck, look for that window behind your other windows.

## What you do

| Session | Notebook part | You will                                                                             |
|---------|---------------|--------------------------------------------------------------------------------------|
| 1       | Part A        | Watch Alice and Bob sign and encrypt in a throwaway sandbox                          |
| 1       | Part B        | Find the primary key, subkey, and self-signatures in your own key                    |
| 1       | Part C        | Compare the instructor's fingerprint with the one on the board, then certify the key |

| Session | Notebook part | You will                                                                                              |
|---------|---------------|-------------------------------------------------------------------------------------------------------|
| 1       | Part D        | Upload <code>my_public_key.asc</code> to Canvas and type your fingerprint into the Canvas quiz        |
| 2       | Part E        | Decrypt your challenge, check the signature, and upload a signed, encrypted <code>response.asc</code> |

## Keep your private key private

- Never upload, email, or commit your private (secret) key. The notebook only exports your **public** key.
- The `.gitignore` blocks `.asc`, `.gpg`, `.pgp`, `.key`, and `.rev` files. Do not override it with `git add -f`.
- Back up your revocation certificate. GnuPG saves it in the `openpgp-revocs.d` folder of your GnuPG home.
- Don't publish your key to a keyserver. This class uses Canvas instead. Mac users: decline any upload offer from GPG Keychain.
- If you think your private key was exposed, tell your instructor right away. You will revoke it and create a new one.

## Troubleshooting

| Problem                                 | Fix                                                                                    |
|-----------------------------------------|----------------------------------------------------------------------------------------|
| <code>gpg not found</code>              | Install Gpg4win or GPG Suite, then restart Jupyter                                     |
| <code>uv: command not found</code>      | Install uv, see <a href="#">installation docs</a>                                      |
| No module named gnupg, or odd errors    | <code>uv pip uninstall -y gnupg</code> , then <code>uv sync</code>                     |
| More than one key pair                  | Delete the extra key in Kleopatra, or set <code>MY_FPR_OVERRIDE</code> in the notebook |
| "unusable public key" or "no assurance" | Run the Part C cell again with the fingerprint from the board                          |
| "No secret key" when decrypting         | Your challenge was made for a different key. Tell your instructor                      |
| Nothing happens                         | The passphrase window may be hidden behind other windows                               |

## Resources

- [Computerphile: Public Key Cryptography](#)
- [Computerphile: What are Digital Signatures?](#)
- [Computerphile: Key Exchange Problems](#)
- [Kevin's Guides: PGP Encryption with Kleopatra](#)
- [Gpg4win documentation](#)
- [python-gnupg documentation](#)